Koord: a language for programming and verifying distributed robotics applications

A robot's code needs to sense the environment, control the hardware, and communicate with other robots. Current programming languages do not provide the necessary hardware platform-independent abstractions, and therefore, developing robot applications require detailed knowledge of signal processing, control, path planning, network protocols, and various platform-specific details. Further, porting applications across hardware platforms becomes tedious.

We present *Koord*—a domain specific language for distributed robotics—which abstracts platform-specific functions for sensing, communication, and low-level control. *Koord* makes the platform-independent control and coordination code portable and modularly verifiable. It raises the level of abstraction in programming by providing distributed shared memory for coordination and port interfaces for sensing and control. We have developed the formal executable semantics of *Koord* in the $\mathbb{K}$ framework. With this symbolic execution engine, we can identify assumptions (proof obligations) needed for gaining high assurance from *Koord* applications. We illustrate the power of *Koord* through three applications: formation flight, distributed delivery, and distributed mapping. We also use the formation flight and distributed delivery applications to demonstrate how platform-independent proof obligations can be discharged using the *Koord* Prover while platform-specific proof obligations can be checked by verifying the obligations using physics-based models and hybrid verification tools.

1 INTRODUCTION

Motivation and goals. Distributed robotic applications (DRAs) could transform manufacturing [Gauthier et al. 1987; Pires and Da Costa 2000], transportation [Gerla et al. 2014; Guo and Yue 2012], agriculture [Blender et al. 2016; R Shamshiri et al. 2018], delivery [Mosterman et al. 2014], and mapping [Thrun et al. 2002]. Following the trends in cloud, mobile, and machine learning applications, programmability is key in unlocking this potential as robotics platforms become more open, and hardware developers shift to the applications marketplace. Available domain specific languages (DSL) for robotics are tightly coupled with platforms, and they combine low-level sensing, communication, and control tasks with the application-level logic (see Section 8 for more details). This tight-coupling and the attendant lack of abstraction hinders application development on all fronts—portability, code reuse, and verification and validation (V&V).

Reliably building a DRA involves two very different concerns: (1) Correctness arguments for coordination algorithms under concurrency and asynchrony are *hardware platform independent*, and use techniques from formal verification and distributed computing. (2) Correctness arguments for physical interactions of the robots (e.g., sensing and motion control) under noise and disturbances are *platform dependent*, and use techniques from control theory. Verification frameworks, such as hybrid automata [Alur and Dill 1994; Henzinger et al. 1995] and hybrid dynamic logic [Platzer 2018], can combine these different types of reasoning at a mathematical level, but are far too abstract for generating executable programs for application of realistic complexity. At the other end, domain specific languages (DSL) for robotics are practical for programming but do not provide

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

, ,

© Association for Computing Machinery.

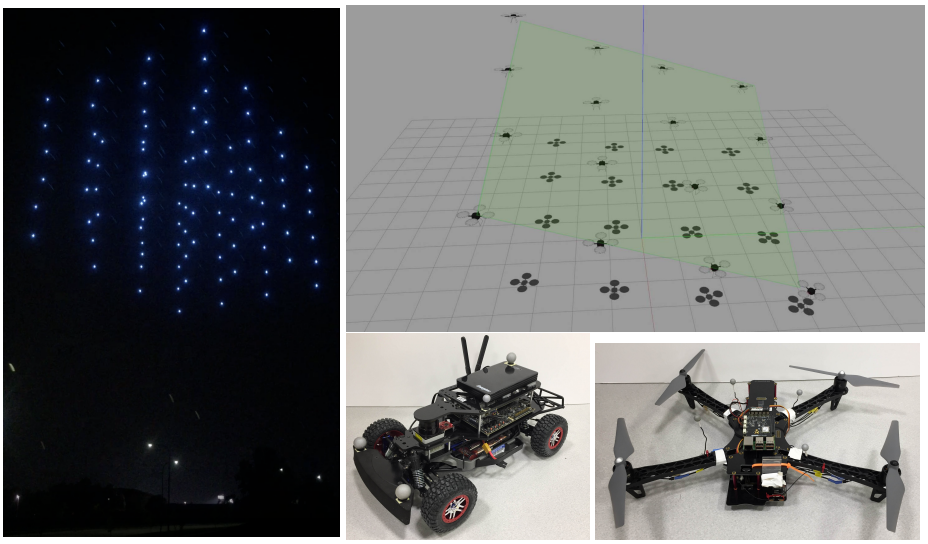

Fig. 2. Swarm formation show by FireFly Inc. (*Left*). A *Koord* application for formation control simulated on 16 virtual drones (*Top Right*). Racing car and quadcopter platforms on which *Koord* applications can be deployed.

precise semantics nor support verification [Blanco 2009; Murali et al. 2019; St-Onge et al. 2017]. The DRONA framework [Desai et al. 2017] bridges this gap, but only in the context of distributed motion planning applications for drones. A more detailed discussion of DRONA and other related works appear in Section 8).

Our Work. We aim to improve reliable engineering of a broader class of DRAs by enabling different types of reasoning at the code level. Our insight is to cleanly decompose the correctness proof of the whole application code into (1) *platform independent proof obligations* for distributed program logic, and (2) *platform dependent proof obligations* for controllers on each target platform. If such decomposition exists, it enables us to plug in analyses from different communities for the different proof-obligations.

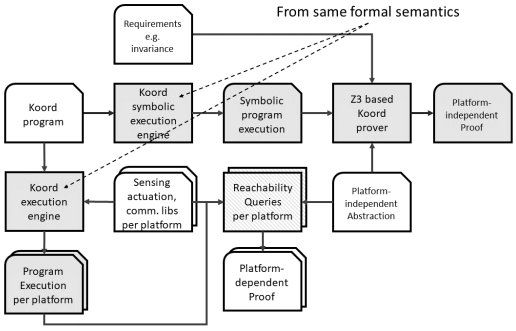

Fig. 1. *Koord* simplifies DRA programming with key abstractions, and tools for verification that can combine different techniques for program logic and platform-specific controllers.

We approach the problem by creating a programming language and a supporting tool chain.

The result is the *Koord* system: a language for DRAs, its formal executable semantics, and supporting verification and testing tools. A user can code DRAs using the *Koord* language. This *Koord* program can be deployed to cars and quadcopters, simulated with virtual vehicles, and verified via our decomposition approach and various existing verification tools. We present the key features of the *Koord* system in this work.

First, *Koord* provides abstractions and language constructs for coordination and control that separate the *platform-independent* program logic, such as distributed decision making, from *platform-dependent* control tasks for sensing, planning, and actuation. This makes *Koord* applications very succinct and readable. A *Koord* program for a set of robots to form a line is only 10 lines (see Figure 3). In another application, robots coordinate and visit waypoints in a mutually-exclusive fashion, while avoiding collisions—all in 50 lines of *Koord* code. A third application, discussed briefly here, accomplishes distributed mapping. Development of these and other nontrivial DRAs, demonstrate the utility of the *Koord* abstractions.

Secondly, we have developed the executable semantics of *Koord* in $\mathbb{K}$ [Rosu and Serbanuta 2014]. To our knowledge this is the first formalization of a programming language for DRAs which has been deployed on actual *heterogenous* platforms. We show that this executable semantics indeed enables us to plug-in different verification techniques for the platform-independent and the platform-dependent proof obligations for the *Koord* applications. We are able to decompose and verify geofencing and collision-avoidance invariants for the above mentioned *Koord* applications. We show that (1) platform-independent proof obligations can be formulated as inductive invariance checks. The invariance checks are further encoded as SMT problems by applying symbolic executions over *Koord* programs, and eventually discharged with Z3. Our experiments show that for upto 15 robots, the time taken for symbolic execution remains relatively stable. The time taken for SMT encoding, and the solving itself increases, but the process completes in the order of seconds using Z3 in Python. This suggests that our verification approach for such proof obligations can scale to multi-robot systems with tens of robots. (2) Platform-dependent proof obligations can be formulated as reachability queries and can be effectively discharged using any number of tools including the simulation-driven reachability tool DryVR [Fan et al. 2017] we used here.

Finally, the correctness of *Koord* still relies on correctly implementing an execution engine whether it's a compiler, a interpreter, or a virtual machine. A consequence of developing our formal semantics of *Koord* in the $\mathbb{K}$ framework, (albeit not a direct contribution of this paper) is that it allows us to generate a reference *verified* interpreter from $\mathbb{K}$. A multi-platform *Koord* execution engine in [Author 2019][1] has been implemented to deploy *Koord* programs to robotic platforms, and program execution on the actual platforms conforms to the formal semantics.

Contributions. In summary, our main contributions are: (1) abstractions to enable separating analyses of platform independent distributed program logic, and platform dependent controllers. (2) a formal executable semantics of *Koord* and case studies demonstrating verification approach and supporting tools (3) a realizable language design (through a formal semantics), which can be deployed on actual hardware platforms.

2 OVERVIEW

We will discuss the key features of the *Koord* programming system with an example application for formation control. This application makes a collection of quadrotors form a pattern as is seen in aerial shows (Figure 2). The *Koord* application LineForm of Figure 3 is a basic version that makes a collection of robots line up uniformly between two extremal robots. Small modifications to the code can make the quadrotors form other shapes like squares, cubes, and stars.

2.1 The *Koord* language

Koord is an event-driven language in which application programs use *shared variables* for coordination across robots and *ports* for interfacing with platform-specific controllers.

[1]This paper has common authors with this work. We have provided an anonymized version with the supplementary material.

```
1  using Motion:
2    sensors: pos psn
3    actuators: pos target
4
5  allread: pos x
6  TargetUpdate:
7    pre True
8    eff if not(pid == N_sys − 1 or pid == 0):
9      Motion.target = mid([x[pid+1],x[pid−1]])
10     x[pid] = Motion.psn
```

Fig. 3. *Koord* program LineForm for a set of robots to form a line.

Port abstractions for platform-dependent control. For the same abstract functions, such as reading the current position, sensing obstacles, and moving from point a to point b in space, different robot platforms need different implementations. *Koord* abstracts the details of these implementations and allows the robot program to interact with its environment through *sensor* and *actuator* ports. For example, LineForm uses a *module* (library) called *Motion* which provides a sensor port called *position* that publishes the robot's position (with some periodicity and accuracy), and an actuator port called *target* for specifying a target position that the controller should try to drive to. Implementations of *Motion* would use different strategies for different platforms. In our experiments, the *Motion* module for a quadcopter uses an indoor camera based positioning system to update the *position* port, and it uses an RRT-based [LaValle 1998] path planner and PID controller. On the other hand, for a small racing vehicle platform, the implementation uses a model-predictive controller [Grüne and Pannek 2017; Kvasnica et al. 2004].

Distributed shared variables for platform-independent coordination. Koord programs can have *shared* variables for participating robots to communicate and coordinate. At Line 5 in Figure 3, the variable x, declared with the **allread** keyword, is a shared array which all robots can read from, but each robot `pid` can only write to x[`pid`]. This shared array makes it possible for a robot to read the current position of other robots in a single line of code. LineForm uses (a) the unique integer identifier `pid` for itself and (b) the number N_{sys} of all participating robots. For multiple robot programs writing to shared variables *Koord* provides concurrency control with mutual exclusion and **atomic** blocks. Semantics of *Koord distributed shared memory (DSM)* is discussed in Section 3.

2.2 Semantics and decomposed verification

In a DRA, multiple instances of the same program are executed by all participants to solve a problem. Execution semantics of such a DRA are complicated by issues of asynchrony, concurrency, as well as the interactions between software and the physical environment. We have developed the full executable semantics of *Koord* in the $\mathbb{K}$ framework [Rosu and Serbanuta 2014]. In solving this problem, we made a few simplifying assumptions: (1) The execution of the *Koord* program advances in a synchronous, *round-by-round* fashion. Each round lasts for some $\delta > 0$ time; δ is a parameter. (2) During a δ-duration round, the robots compute, move, and communicate with each other through distributed shared memory. While these assumptions sidestep issues of asynchrony and failures, they make our executable semantics tractable, and experiments show that these assumptions are met by the platforms in question [Author 2019].

Koord's executable semantics enables explicit and exhaustive exploration of the non-deterministic behaviors of *Koord* applications. We have also implemented a *Koord* Prover tool on top of these semantics for symbolically checking inductive invariants for *Koord* programs. Consider a natural requirement for LineForm which is Geofencing: given a (hyper)rectangle *rect*(a, b), defined by two

corners a and b, if all robots are initialized within $rect(\mathrm{a}, \mathrm{b})$, then they stay in $rect(\mathrm{a}, \mathrm{b})$ at all times. This requirement can be stated as an invariant of the system:

Invariant 1. $\bigwedge_{i \in \mathtt{ID}} \left(\; Motion.psn_i \in rect(\mathrm{a}, \mathrm{b}) \wedge x_i \in rect(\mathrm{a}, \mathrm{b}) \; \right)$

Checking Invariant 1 requires reasoning about both platform-dependent and independent parts of the application. Using *Koord* tools one can reason about it in a decomposed fashion:

(1) Assuming that all shared position $x[i]$ are in $rect(\mathrm{a}, \mathrm{b})$, we have to show that the targets computed by LineForm are in $rect(\mathrm{a}, \mathrm{b})$. This platform-independent proof obligation is about the correctness of the program logic of LineForm. To check this, one has to compute the post states of the *TargetUpdate* event and check that Invariant 1 is closed under this post operation. The *Koord* Prover uses the symbolic semantics for post computation and encodes this check as an SMT problem. In case of Invariant 1, and many other applications and invariants, this proof obligation is discharged fully automatically.
(2) Assuming that the sensed current position $Motion.psn_i$ and the computed target are in $rect(\mathrm{a}, \mathrm{b})$, we have to show that a given robot's controller indeed keeps it in $rect(\mathrm{a}, \mathrm{b})$. This platform-dependent proof obligation is about the correctness of the controller implemented in the *Motion* module. *Koord* helps identify these obligations or assumptions about *Module* implementations. If we have a model *traj* to compute trajectory of the vehicle's motion, we can state and prove this as:

Assumption 1.

$$\forall t \in [0, \delta], traj(Motion.psn, Motion.target, t) \subseteq rect(Motion.psn, Motion.target),$$

where *traj* gives the position of the robot at time t, as a function of the target and initial position at the beginning of the round. To check these types of assumptions, we can use a reachability analysis tool for dynamical and hybrid systems, of which there are many [Bak and Duggirala 2017; Chen et al. 2013; Duggirala et al. 2013; Fan et al. 2017; Frehse et al. 2011]. In our experiments we use the simulation-driven reachability tool DryVR [Fan et al. 2017] which is scalable and provides probabilistic guarantees, but does not require complete dynamical models.

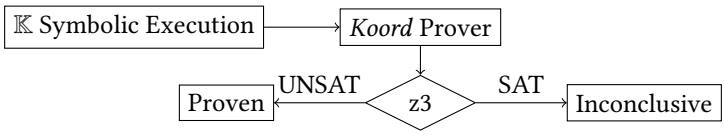

Fig. 4. *Koord* Prover uses 𝕂 -based executable semantics and z3 for checking invariants.

2.3 *Koord* Compiler, Implementation and Simulator

In this paper, we present the *Koord* design, semantics and associated formal analysis techniques, but *not* the implementation of the language and system. An open source implementation of *Koord* is presented in [Author 2019], which includes programming tools for simulation, and hardware deployment. *Koord* applications implemented using the aforementioned programming tools have been deployed on heterogeneous multi-robot systems of quadcopters and small racing vehicles.

The implementation of those programming tools is not part of the contributions of this paper. This implementation includes a compiler (written using Antlr4 in Java) for *Koord* as well. The compiler performs straightforward code generation of application code from abstract syntax trees, which in turn are generated from from the BNF grammar of the *Koord* language presented in Section 3.3.

2.4 Engineering reliable DRAs

Koord tools support the engineering of reliable systems by helping discover and validate platform-dependent proof obligations (see case studies in Section 5 and Section 6). In general, if the assumptions needed for proving correctness of an application are too strong, a DRA engineer could either revise the assumptions or modify the invariant requirement so that weaker assumptions may be sufficient. Using the high-fidelity *Koord* simulator which is part of the *Koord* programming tools, we can gain insights about when such assumptions are violated.

For instance, we see in Section 5 that sensitivity analysis using DryVR is able to detect violations of Assumption 1. A quadcopter model with poor PID control could temporarily go out of bounds due to inertia while moving towards the target. Upon configuring the same quadcopter model with different PID control parameters, DryVR was able to verify Assumption 1. Similarly, DryVR is able to detect that a ground vehicle may not be able to follow a path computed by a path-planner as closely as required for maintaining safe distances between vehicles in Section 6. As we shall see in these case studies, these assumptions require reasoning only about the platform-dependent control ports, allowing us to decouple their verification from the distributed program logic.

3 THE *KOORD* LANGUAGE

In this section, we present the syntax and the semantics of *Koord*. When a *Koord* application is deployed on a fleet of N_{sys} robots, each robot runs an instance of the same program. There is a known set of identifiers $\texttt{ID} = \{0, 1, \ldots, N_{sys} - 1\}$, and each robot is assigned a unique index $\texttt{pid} \in \texttt{ID}$.

3.1 Syntax

Figure 5 shows the core grammar of *Koord* syntax. Each robot program consists of (a) declarations of the interfaces between the program and the sensor/actuator modules, (b) declarations of shared and local program variables, and (c) events, consisting of preconditions and effects. Robot programs (rule *Program*) first can import sensor/actuator modules. The module import grammar production specifies the interfaces or ports: it contains all input and output ports for actuators (*APorts*) and sensors (*SPorts*) that the program uses. To summarize, there are following three types of names for reading/writing values:

(i) *Sensor and actuator ports* are used to read from sensor ports and write to actuator ports of controllers.
(ii) *Local program variables* record the state of the program.
(iii) *Distributed shared variables* are used for coordination across robots. All shared variables can be read by all participating robots; an **allwrite** variable can be written by any participating robot; while an **allread** variable can be written only by a single robot.

Users can then optionally specify the initial values of program variables (rule *Init*). The main body of the program is a sequence of events (rule *Event*) which include a Boolean precondition (**pre**) and and an effect (**eff**). The effect of an event is also a statement (rule *Effect*).

A statement (rule *Stmt*) in *Koord* resembles those in most imperative languages and includes conditional statements, function calls, assignments, blocks of statements, etc. Mutual exclusion is always an essential feature when shared variables are involved. *Koord* provides a locking mechanism using the keyword **atomic** to update the shared variable safely. The user can also define functions and abstract data types (tuples of the basic data types). These features enable a natural separation of the discrete computational (platform independent) and dynamic (platform dependent) behaviors. To discuss these behaviors, we need to establish the notion of system and robot state.

```
Program   ::= Defs Module+? DeclBlock Init? Event+

Defs      ::= [FuncDef+]? [AdtDef+]?
FuncDef   ::= def fun identifier(Param+) : Stmt+
AdtDef    ::= def adt identifier : Decl+
Param     ::= Type identifier

Module    ::= using module identifier : SPorts APorts
SPorts    ::= sensors : Decl+
APorts    ::= actuators : Decl+

Decl      ::= Type identifier | Type identifier =Val
Type      ::= int | float | bool | pos | adt
            | Type[Int] | List ⟨Type⟩ | Queue ⟨Type⟩

DeclBlock ::= AWDecls ARDecls LocalDecls
AWDecls   ::= allwrite : Decl+
ARDecls   ::= allread : Decl+
LocalDecls ::= local : Decl+

Init      ::= init : Stmt+
Event     ::= identifier : pre (Cond) eff : Stmt+

Expr      ::= AExpr|BExpr
AExpr     ::= AExpr AOp AExpr
            | Expr++ | -AExpr | Var | AVal
AOp       ::= + | − | ∗ | /
BExpr     ::= Expr RelOp Expr|Expr COp Expr
            | notExpr | Var | BVal
RelOp     ::= ≥|≤|≥|==|>|<|≠
COp       ::= and | or

Stmt      ::= Assign | FnCall | Atomic
            | Ite | Loop | Return
Assign    ::= Var = Expr
Ite       ::= if BExpr Stmt+ [else Stmt+]?
FnCall    ::= identifier(Expr+)
Atomic    ::= atomic : Stmt+
Loop      ::= for identifier in AExpr : Stmt+
Return    ::= return Expr | return

Var       ::= identifier | identifier[Expr]
            | identifier.identifier
Val       ::= AVal | BVal
AVal      ::= Int | Float
BVal      ::= Bool
```

Fig. 5. Core *Koord* program syntax.

3.2 Configurations

Recall from Section 2 the semantics of a *Koord* program execution is based on synchronous rounds. Each round is divided into *program transitions* and *environment transitions* that update the *system configuration*. In each round, each robot performs at most one event. The update performed by a single robot executing an event is modeled as an instantaneous transition that updates the program variables and potentially actuator ports; however, different events executed by different robots may interleave in an arbitrary order. In between the events of successive rounds, $\delta > 0$ duration of time elapses, the program variables remain constant while the values held by the sensor and actuator ports may change. These are modeled as environment transitions that advance time as well as the sensor and actuator ports. Thus, each round consists of a burst of (at most N_{sys}) program transitions followed by an environment transition. This is a standard model for synchronous distributed systems where the speed of computation is much faster than the speed of communication and physical movement [Attiya and Welch 2004; Lynch 1996].

We now describe the system state, or *system configurations* which we use to formalize *Koord* semantics.

System configurations. A *system configuration* is a tuple $\boldsymbol{c} = (\{L_i\}_{i \in \mathtt{ID}}, S, \tau, \mathit{turn})$, where

(i) $\{L_i\}_{i \in \mathtt{ID}}$ or $\{L_i\}$ in short is an indexed set of *robot configurations*–one for each participating robot. L_i refers to the configuration of the i-th element, i.e., the i-th robot in the system.
(ii) $S : \mathit{Var} \mapsto \mathit{Val}$ is the *global context*, mapping all shared variable names to their values.
(iii) $\tau \in \mathbb{R}_{\geq 0}$ is the *global time*.
(iv) $\mathit{turn} \in \{\mathtt{prog}, \mathtt{env}\}$ is a binary *bookkeeping* variable determining whether program or environment transitions are being processed.

Bookkeeping variables are invisible in the language syntax, and only used in the semantics. Note that, while *turn* for the system configuration may be a bookkeeping variable, it is directly used to achieve the separation of platform dependent and platform independent concerns in the semantics. We now define the robot configurations which define the state of every robot in the system.

$$\frac{\forall i \in \texttt{ID}, L_i.turn = \texttt{env}}{(\{L_i\}, S, \tau, \texttt{prog}) \rightarrow_G (\{L_i\}, S, \tau, \texttt{env})} \quad \textsc{EndProgTrans}$$

$$\frac{\begin{array}{c}\forall i \in \texttt{ID}, \langle S, L_i\rangle \rightarrow_{env} \langle S, L_i'\rangle \\ \wedge\, L_i.turn = \texttt{env} \wedge L_i'.turn = \texttt{prog}\end{array}}{(\{L_i\}, S, \tau, \texttt{env}) \rightarrow_G (\{L_i'\}, S, \tau + \delta, \texttt{prog})} \quad \textsc{EnvTrans}$$

$$\frac{\begin{array}{c}\exists i \in \texttt{ID}, \langle S, L_i, \oplus\rangle \rightarrow_{stmt} \langle S', L_i', \cdot\rangle \\ \wedge\, L_i.turn = \texttt{prog} \wedge L_i'.turn = \texttt{env}\end{array}}{(\{L_i\}, S, \tau, \texttt{prog}) \rightarrow_G (\{L_i'\}, S', \tau, \texttt{prog})} \quad \textsc{EventTrans}$$

Fig. 6. System semantic rules for *Koord*.

Robot configurations. A *robot configuration* is used to specify the semantics of each robot. As a *Koord* program is run on a system of robots, each participating robot would have its own set of module ports and local variables, along with a local copy of each shared variable. Given a *Koord* program P, we can define *Var* be the set of variables, *Val* be the set of values that an expression in *Koord* can evaluate to, *CPorts* be the set of sensor and actuator ports of the controller being used, and *Events* the set of events in P. A robot configuration is a tuple $L = (M, cp, turn)$, where

(i) $M : Var \mapsto Val$ is its *local context* mapping both local and shared variables to values. Note that this implies M includes a copy of shared variable values.
(ii) $cp : CPorts \mapsto Val$ is the mapping of sensor and actuator ports to values.
(iii) $turn \in \{\texttt{prog}, \texttt{env}\}$ is a bookkeeping variable indicating whether this robot is executing a program or environment transition.

For readability, we use the dot (".") notation to access components of a robot configuration L. For example, $L.M$ means accessing the local context M in the tuple L.

3.3 Semantics

The execution semantics for a *Koord* program captures the separation of the platform independent distributed program behaviors and the platform specific controller behaviors (the program and environment transitions) of the robots through *rewrite rules*. Rewrite rules at various levels : *System*, *Robot* and *Expression* are required to specify the semantics of a *Koord* program, and allow us to create a framework for formal analysis.

System semantics. For system-level semantics, the rewrite rule is a mapping from an initial system configuration to a set of configurations. It has the type

$$\rightarrow_G \subseteq \mathbb{C} \mapsto \wp(\mathbb{C}),$$

where $\mathbb{C}$ is the set of all possible system configurations.

The bookkeeping variable *turn* is used by the system to determine whether the system (all robots in the system) is performing a program transition, or an environment transition. The system executes an environment transition only when the local *turn* of each robot is env. After all robots enter the env turn, rule EndProgTrans sets the global *turn* from prog to env indicating the end of program transition, and an environment transition will occur afterwards.

Rule EnvTrans shows the evolution of the system configuration after the rule EndProgTrans is applied.This rule synchronizes the *environment* transitions of the robots and advances the global time from τ to $\tau + \delta$.

During a program transition, each robot executes a sequence of statements, or rewrite rules for statement semantics of type

$$\rightarrow_{stmt} \subseteq (\mathbb{S} \times \mathbb{L} \times (Stmt \cup \{\oplus, \cdot\})) \mapsto \wp(\mathbb{S} \times \mathbb{L} \times Stmt \cup \{\cdot\}),$$

$$\frac{\begin{array}{l} L.turn = \texttt{prog} \\ \wedge \text{“}Name\text{: } \textbf{pre:} \ Cond \ \textbf{eff:} \ Body\text{”} \in Events \wedge [\![Cond]\!]_{S,L} \end{array}}{\langle S, L, \oplus \rangle \rightarrow_{stmt} \langle S, L, Body \rangle} \ \textsc{SelectEvent}$$

$$\langle S, L, \oplus \rangle \rightarrow_{stmt} \langle S, L, \cdot \rangle \ \textsc{SkipEvent} \qquad \langle S, (M, cp, \texttt{prog}), \cdot \rangle \rightarrow_{stmt} \langle S, (M, cp, \texttt{env}), \cdot \rangle \ \textsc{EndEvent}$$

$$\frac{\forall x \in Keys(S), M' = M[x \mapsto S[x]] \wedge cp' = f(cp, \delta)}{\langle S, (M, cp, \texttt{env}) \rangle \rightarrow_{env} \langle S, (M', cp', \texttt{prog}) \rangle} \ \textsc{RobotEnv}$$

Fig. 7. Partial per robot semantic rules for *Koord*.

where *Stmt* refers to the set of all possible statements allowed by *Koord* syntax. We use internal syntactic structures '$\oplus$' and '$\cdot$', which are are not in *Koord* themselves, but are used to represent control flow in *Koord* programs in the semantics, as we will see in the discussion on per-robot semantics. '$\oplus$' is to denote nondeterministic selection of events, and '$\cdot$' is to indicate an "empty" statement.

This relation takes as input a tuple of (1) a global context, (2) a robot configuration, and (3) a statement, and maps it to a set of such tuples. Rule EventTrans expresses that starting from a system configuration $\boldsymbol{c} = (\{L_i\}, S, \tau, \texttt{prog})$, a robot i with the configuration L_i starts by selecting an enabled event, executes the event via a sequence of $\rightarrow_{stmt}$ rewrites, and sets its own *turn* to env at the end of the event execution. The system goes from a configuration $\boldsymbol{c}$ to $\boldsymbol{c}' = (\{L_i'\}, S', \tau, \texttt{prog})$, with possibly different robot configurations and global context depending on whether any statement executed resulted in writes to shared variables. The system can display nondeterministic behaviors arising from different robots executing their events in different orders.

We now go into some detail to discuss the $\rightarrow_{stmt}$ rewrites which specify the behavior of each robot during a program transition. These rules are used to individual *robot* configurations.

Per robot semantics. Events are the main computational blocks in a *Koord* program. We present the core semantic rules for event execution by a robot executing a *Koord* program. Rule SelectEvent in Figure 7 shows that any event may be executed when the precondition *Cond* is evaluated to true, and by replacing $\oplus$ with the event effect *Body*, it ensures only one event is selected and executed. The event effect is then executed following the semantics of each statement in *Body*. Rule SkipEvent allows the robot to skip the event completely. At the end of the event, the sequence of statements becomes empty '$\cdot$'. Rule EndEvent then makes sure the *turn* of the robot is set to env indicating that an environment transition will occur afterwards.

While $\rightarrow_{stmt}$ rewrites define each robot's behavior during a program transition, we separate the platform-dependent semantics of how each robot interacts with environment (including other robots) using environment transition rules of the type

$$\rightarrow_{env} \subseteq (\mathbb{S} \times \mathbb{L}) \mapsto \wp(\mathbb{S} \times \mathbb{L}),$$

which takes a global context and a robot configuration as input. Rule RobotEnv simply states that the new local context M' is the old local context M updated with the global context S; thus ensuring that all robots have consistent shared variable values before the next program transition.

To define the executable $\mathbb{K}$ semantics of *Koord* applications, we have to provide executable descriptions for the environment transitions. The type of this executable object (f) is defined by *CPorts*, namely, $f : [CPorts \mapsto Val] \times \mathbb{R}_{\geq 0} \mapsto [CPorts \mapsto Val]$. That is, given old sensor and actuator values and a time point, f should return the new values for all sensor and actual ports.

New sensor readings cp' are then obtained by evaluating the black-box dynamics f with time δ. In an actual execution, the controller would run the program on hardware, whose sensor ports evolve for δ time between program transitions. This formalization allows the ports to behave

$$\frac{\langle S, L, St\rangle \rightarrow_{stmt} \langle S', L', St'\rangle}{\langle S, L, St\ StList\rangle \rightarrow_{stmt} \langle S', L', St'\ StList\rangle}\ \textsc{StmtSeq1}$$

$$\langle S, L, \cdot\ StList\rangle \rightarrow_{stmt} \langle S, L, StList\rangle\ \textsc{StmtSeq2}$$

$$\frac{x \in Keys(S) \wedge x \in Keys(L.M) \wedge L'.M = L.M[x \mapsto v]}{\langle S, L, x = v\rangle \rightarrow_{stmt} \langle S[x \mapsto v], L', \cdot\rangle}\ \textsc{SvarAssign}$$

$$\frac{x \notin Keys(S) \wedge x \in Keys(L.M) \wedge L'.M = L.M[x \mapsto v]}{\langle S, L, x = v\rangle \rightarrow_{stmt} \langle S, L', \cdot\rangle}\ \textsc{LvarAssign}$$

Fig. 8. Example statement level semantic rules for *Koord*.

arbitrarily over δ-transitions. Hence in verification, additional assumptions over the behavior of the sensor and actuator ports are needed. Finally, the *turn* of the robot is set back to `prog`.

Aside from such rules,during program transitions, *Koord* semantics include rewrite rules showing the impact of the shared memory abstractions on the configurations of each of the robot, control flow, etc. We illustrate a few of these rules, in Figure 8 with rules such as Rule StmtSeq1 and StmtSeq2 showing how a statement representing a sequence of statements is executed. Rule LvarAssign and Rule SvarAssign show the semantic rules for local and shared variable assignment respectively are also examples of statement level rules. Evaluating these rules requires in turn, expression-level rules, which include variable lookup, arithmetic, boolean , and relational operations amongst others. We present a few illustrative examples below.

Expression-level Semantics. The expression level semantics are given by rewrite rules of the type

$$\rightarrow_E \subseteq (\mathbb{S} \times \mathbb{L} \times \mathbb{E}) \times (\mathbb{S} \times \mathbb{L} \times \mathbb{E}),$$

where $\mathbb{S}$ is the set of all possible global contexts S, $\mathbb{L}$ refers to the set of all possible values for configurations of an robot, and $\mathbb{E}$ refers to the set of all possible expressions allowed by the language syntax.

The variable lookup rule Var-Lookup-Rule states that every robot has a local copy of every variable in the program, and if an robot is evaluating an expression involving variable x, it will replace x with the current value v from the local context M. $M[x]$ here obtains the value corresponding to the key x.

We also present the rules for addition (Aexpr-Add-rule). They are fairly standard: the execution first evaluates the left subexpression, then the right subexpression given that left is already evaluated to a value and finally adding the two values. We omit the similar rules for other arithmetic, boolean, and relational operations.

$$\frac{L.M[x] = v}{\langle S, L, x\rangle \rightarrow_E \langle S, L, v\rangle}\ \textsc{Var-Lookup-rule}$$

$$\frac{E_1 \rightarrow_E E_1'}{\langle S, L, E_1 + E_2\rangle \rightarrow_E \langle S, L, E_1' + E_2\rangle}\ \textsc{Aexpr-Add-rule-1}$$

$$\frac{E_1 \in Val \wedge E_2 \rightarrow_E E_2'}{\langle S, L, E_1 + E_2\rangle \rightarrow_E \langle S, L, E_1 + E_2'\rangle}\ \textsc{Aexpr-Add-rule-1}$$

$$\frac{E_1 \in Val \wedge v_1 + v_2 \rightarrow_E v_3}{\langle S, L, v_1 + v_2\rangle \rightarrow_E \langle S, L, v_3\rangle}\ \textsc{Aexpr-Add-rule-2}$$

Fig. 9. Partial expression semantic rules for *Koord*.

The semantic rules we discussed realize the distributed nature of the design of the *Koord* system. The memory consistency model, and the synchronization model of *Koord* have been designed to complement the separation and analysis of the platform independent program transitions and platform dependent environment transitions.

3.4 Synchronization and consistency

Following our semantic rules in Section 3.3, careful readers would notice that all program transitions of *Koord* program take *zero* time. The environment transitions however take δ time for the evolution of the sensor and actuator ports together with the update of the local context from the global context.

To reiterate, the following are the timing requirements from rule EventTrans and EnvTrans: (a) a program transition takes *zero* time, (b) new values of controller ports are sampled at the end of each round (c) shared variables should reach consistent values within δ time, and (d) a global clock is used to synchronize each δ-time round. The first two requirements are achievable if the time taken to complete a program transition is negligible compared to δ, and δ can be a common multiple of the sampling intervals of all controller ports in use. These constraints are reasonable when computation and communication is comparatively much faster. Using the *Motion* module as an example, our position sensor on each device publishes every 0.01 sec (100Hz) while the CPU on each drone is 1.4 GHz. If we set δ to be 0.01 sec, a program transition taking 10K CPU cycles is still less than 0.1% of δ.

Requirement (c) and (d) are common research questions in distributed computing with an extensive literature. A global clock can be achieved with existing techniques that synchronize all local clocks on robots. The toolchain in [Author 2019] uses message passing to implement distributed shared memory for shared variables. It ensures that the time taken to propagate values through messages and reach consistency is smaller than δ, and the update is visible in the next round of program transitions for all robots. We therefore conclude our round based semantic with shared memory is a reasonable abstraction.

4 VERIFYING *KOORD* PROGRAMS

We have built the semantics of *Koord* in the $\mathbb{K}$ framework to enable decoupled analyses of platform-independent distributed program logic and the platform-dependent controllers of DRAs. The *events* in an *Koord* program define the distributed program logic in the system. The effect of a robot i executing event $e \in Events$ on a configuration $\boldsymbol{c} \in \mathbb{C}$, can be seen as a $\rightarrow_{stmt}$ application to $\langle \boldsymbol{c}.S, \boldsymbol{c}.L_i, Body \rangle$, where e is *"eventName:* **pre:** *Cond* **eff:** *Body"*.

4.1 Reachable configurations

Given a set of system configurations C, we define the following functions using the semantic rules of Section 3.3: (i) C_i^e returns the set of configurations obtained by robot i executing event $e \in Events$ from a configuration in C. (ii) C_i^+ returns the set of configurations obtained by robot i executing any event from a configuration in C. (iii) $C_{\vec{p}}^+$ returns all configurations visited, when robots execute their events in the order $\vec{p}$, where $\vec{p}$ is a sequence of $p_i \in \mathtt{ID}$. (iv) C^+ is the union of $C_{\vec{p}}^+$ over all orders $\vec{p}$. (v) $C_{\mathtt{prog}}^+$ is the set of configurations reached from C *after* a program transition. All definitions can be restricted naturally to individual configurations.

$$
\begin{aligned}
C_i^e &:= \{\boldsymbol{c}' \mid \exists \boldsymbol{c} \in C, [\![Cond]\!]_{\boldsymbol{c}.S, \boldsymbol{c}.L_i} \\
&\qquad \wedge \langle \boldsymbol{c}.S, \boldsymbol{c}.L_i, Body \rangle \rightarrow_{stmt} \left\langle \boldsymbol{c}'.S, \boldsymbol{c}'.L_i, \cdot \right\rangle\}, \\
C_i^{Skip} &:= \{\boldsymbol{c}' \mid \exists \boldsymbol{c} \in C, \langle \boldsymbol{c}.S, \boldsymbol{c}.L_i, \cdot \rangle \rightarrow_{stmt} \left\langle \boldsymbol{c}'.S, \boldsymbol{c}'.L_i, \cdot \right\rangle\} \\
C_i^+ &:= C_i^{Skip} \cup \bigcup_{e \in Events} (C_i^e), \\
C_{\vec{p}}^+ &:= \begin{cases} \emptyset, & \text{if } \vec{p} = () \\ (C_{p_0}^+)_{\vec{p}'}^+, & \text{if } \vec{p} = (p_0, \vec{p}') \end{cases} \\
C^+ &:= \bigcup_{\vec{p} \in perms(\mathtt{ID})} C_{\vec{p}}^+, \\
C_{\mathsf{prog}}^+ &:= \left\{ \boldsymbol{c} \mid \boldsymbol{c} \in C^+ \wedge \forall i \in \mathtt{ID}, \boldsymbol{c}.L_i.turn \neq \mathsf{prog} \right\}.
\end{aligned}
$$

In the above, a sequence $\vec{p} = (p_0, \vec{p}')$, is written as a concatenation of the first element p_0 and the suffix $\vec{p}'$. Also, $perms(\mathtt{ID})$ refers to the set of permutations of $\mathtt{ID}$.

Next, we define the configurations that are reached during and after an environment transition. Recall that environment transitions capture the evolution of the sensor and actuator ports over a time interval $[0, \delta]$—all other parts of the configuration remain unchanged. Our *Koord* semantics defines the environment transitions with a *parameter* which is a (possibly black-box) function that captures the dynamics of individual robots.[2] Given such a function f_i for each robot i, we define the function $traj : \mathbb{C} \times [0, \delta] \mapsto \mathbb{C}$ to represent the evolution of the system over a $[0, \delta]$ time interval. $traj$ is constructed by simply updating all controller ports cp of each robot i using the function f_i that captures their respective dynamics. That is,

$$
\boldsymbol{c}' = traj(\boldsymbol{c}, t) \Leftrightarrow \begin{pmatrix} \forall i \in \mathtt{ID}, \boldsymbol{c}'.L_i.cp = f_i(\boldsymbol{c}.L_i.cp, t) \\ \wedge\ \boldsymbol{c}'.L_i.M = \boldsymbol{c}.L_i.M \\ \wedge\ \boldsymbol{c}'.L_i.turn = \boldsymbol{c}.L_i.turn \\ \wedge\ \boldsymbol{c}'.S = \boldsymbol{c}.S \wedge \boldsymbol{c}'.\tau = \boldsymbol{c}.\tau \\ \wedge\ \boldsymbol{c}'.turn = \boldsymbol{c}.turn \end{pmatrix}
$$

Notice those additional constraints making sure all other fields of $\boldsymbol{c}$ and $\boldsymbol{c}'$ stay the same. The set of all transient system configurations $C_{[0,t]}$ reached in an interval $[0, t]$ from C:

$$
C_{[0,t]} := \{\boldsymbol{c}' \mid \exists \tau \in [0, t], \exists \boldsymbol{c} \in C, \boldsymbol{c}' = traj(\boldsymbol{c}, \tau)\}.
$$

We denote the set of points reached exactly at the end of an environment transition from C as C_{env}.

$$
C_{\mathsf{env}} := \{\boldsymbol{c}' \mid \exists \boldsymbol{c} \in C, \boldsymbol{c}' = traj(\boldsymbol{c}, \delta)\} \text{ where } \delta \text{ is the time for a round.}
$$

Now to conform to our semantics, we carefully define the exact set of configurations reached right at the end of each *round* without transient configurations. A *frontier* set of configurations C^n represents those configurations that are reached from C *exactly when* n rounds are completed. Formally,

$$
C^n := \begin{cases} C, & \text{if } n = 0 \\ ((C^{n-1})_{\mathsf{prog}}^+)_{\mathsf{env}} & \text{otherwise} \end{cases}
$$

Finally, given a set of configurations $C \subseteq \mathbb{C}$, the set of all reachable states in n rounds is defined inductively:

$$
Reach(C, n) := \begin{cases} C, & \textit{if } n = 0 \\ Reach(C, n-1) \cup\ (C^{n-1})^+ \cup ((C^{n-1})_{\mathsf{prog}}^+)_{[0,\delta]}, & \text{otherwise} \end{cases}
$$

[2]For different platforms, this function could be defined in closed form, as solutions of differential equations, or in terms of a numerical simulator.

Notice that all transient configurations during both program and environment transitions are included in *Reach*.

4.2 Decomposing invariance verification

An *invariant* of a *Koord* program is a predicate that holds in all reachable configurations. Invariant requirements can express safety, for instance, that no two robots are ever too close (Collision avoidance), or that robots always stay within a designated area (Geofencing). Formally,

Definition 4.1. An invariant *inv* is a predicate (Boolean valued function) over a configuration or set of configurations such that, given a set of initial configurations of the system C_0,

$$\forall n \in \mathbb{N}, [\![inv]\!]_{Reach(C_0,n)}$$

where $[\![inv]\!]_C$ represents evaluating *inv* over each configuration in C. We use the notation $[\![inv]\!]_{\boldsymbol{c}}$ for evaluating *inv* over a single configuration $\boldsymbol{c}$ as well.

Definition 4.2. A predicate *inv* is an *inductive invariant* of a system if given a set of initial configurations of the system C_0, the following proof obligations (POs) hold:

$$[\![inv]\!]_{C_0} \tag{Base}$$

$$[\![inv]\!]_C \Rightarrow [\![inv]\!]_{Reach(C,1)} \tag{1}$$

That is, *inv* holds in the initial configuration(s) (PO (*Base*)), and *inv* is preserved by both platform-independent program transitions (distributed program logic) and the platform-dependent environment transitions (controllers). according to PO (1). It is straightforward to prove that an inductive invariant is an invariant of the system.

Our verification strategy for user-specified (inductive) invariants is to discharge the proof obligations. PO (*Base*) is usually trivial. Therefore, we focus on PO (1). The *Koord* semantics enables us to *decouple* the environment and program transitions in *Reach*, and analyze each separately. PO (1) can be restated as

$$[\![inv]\!]_C \Rightarrow [\![inv]\!]_{C^+} \tag{2}$$

$$[\![inv]\!]_C \Rightarrow [\![inv]\!]_{(C^+_{\mathsf{prog}})_{[0,\delta]}} \tag{3}$$

As in other concurrent systems, a major bottleneck in computing C^+ for PO (2) is the required enumeration of all $\vec{p} \in perms(\mathtt{ID})$ permutations for all robots with reads/writes to shared contexts. We therefore seek for a stronger and easier to prove proof obligation using the lemma below:

Lemma 4.3. *Given any inductive predicate φ, for any configuration c satisfying φ, the following always holds*

$$\left(\bigwedge_{i \in \mathtt{ID}} \bigwedge_{e \in Events} [\![\varphi]\!]_{C_i^e} \right) \Rightarrow [\![\varphi]\!]_{C^+}$$

The proof follows from expanding the definition of C^+ and inducting on each event sequence. As φ is preserved before and after every event transition of every robot, the order of robot events do not violate φ. With Lemma 4.3, we strengthen and rewrite PO (2) as

$$[\![inv]\!]_C \Rightarrow \bigwedge_{i \in \mathtt{ID}} \bigwedge_{e \in Events} [\![inv]\!]_{C_i^e} \tag{4}$$

which no longer requires enumeration of all permutations. This is how our synchronous model of execution helps verification scale.

We now discuss our approach to discharge PO (3). To further decouple program and environment transitions, we rewrite PO (3) by expanding $[\![inv]\!]_{(C^+_{\mathsf{prog}})_{[0,\delta]}}$ and derive:

$$[\![inv]\!]_C \Rightarrow (\forall \boldsymbol{c}', \boldsymbol{c}'', \forall t \in [0,\delta], \boldsymbol{c}' \in C^+_{\mathsf{prog}} \wedge \boldsymbol{c}'' = traj(\boldsymbol{c}', t) \Rightarrow [\![inv]\!]_{\boldsymbol{c}''}). \tag{5}$$

PO (5) requires reasoning about the dynamic behavior of *traj* during environment transitions, and it is a challenging research problem by itself. We introduce *controller assumption* to abstract away the continuous dynamic behavior.

Definition 4.4. A *controller assumption* is a pair of predicates $\langle P, Q\rangle$, where P is defined over $CPorts \times Val \times CPorts \times Val$ and Q is over $CPorts \times Val$. Given a *controller assumption* $\langle P, Q\rangle$, the *traj* function satisfies the assumption if starting from any $\boldsymbol{c}'$ with port values satisfying P then any reachable configuration $\boldsymbol{c}''$ within $[0, \delta]$ must satisfies Q. Formally,

$$\forall \boldsymbol{c}', \boldsymbol{c}'', \forall t \in [0, \delta], P(\boldsymbol{c}'.Acts, \boldsymbol{c}'.Sens) \wedge \boldsymbol{c}'' = traj(\boldsymbol{c}', t) \Rightarrow Q(\boldsymbol{c}''.Sens) \qquad (AAsm)$$

where $\boldsymbol{c}'.Acts$ refers to its actuator port values, $\boldsymbol{c}'.Sens$ refers to the sensor port values, A controller assumption $\langle P, Q\rangle$ is similar to preconditions and postconditions for the *traj* function with an additional guarantee that Q must hold at all time during the time horizon $[0, \delta]$. It allows users to over-approximate the set of all transient configurations reached by *traj* and prove the invariant. We demonstrate in Section 5 and Section 6 how controller assumption can be validated with specialized tools for continuous dynamics.

We know by definition $C^+_{\mathsf{prog}} \subseteq C^+$. With Lemma 4.3, we can merge PO (4) and PO (5), add program and controller assumptions, and simplified our proof obligation as:

$$\bigwedge_{i \in \mathrm{ID}} \bigwedge_{e \in Events} [\![inv]\!]_C \wedge \boldsymbol{c}' \in C^e_i \wedge (P(\boldsymbol{c}'.Acts, \boldsymbol{c}'.Sens) \Rightarrow Q(\boldsymbol{c}''.Sens)) \Rightarrow [\![inv]\!]_{\boldsymbol{c}''} \qquad (Ind)$$

Notice the continuous dynamics no longer appear in PO (*Ind*), and it allows us to reason in per event fashion as well as per robot fashion. We then can use our $\mathbb{K}$ symbolic execution semantics to construct the symbolic post configurations C^e_i for each event e, and prove the validity with SMT solvers.

Dealing with loops and module functions. Many programs can include module related functions, and loops. Proving invariants over such loops is by itself a well studied and difficult research problem. To mitigate this problem, we instruct our symbolic execution to treat module related functions such as computing distance between two points, and paths generated by path planners as *uninterpreted functions*, and we introduce *function summary* for these uninterpreted functions similar to controller assumptions.

Definition 4.5. A *function summary* $F(x, y)$ for an uninterpreted function $f(x)$ is a predicate which we can prove the following proof obligation:

$$\forall x, F(x, f(x)) \qquad (FSum)$$

where x can be extended according to the arity of f. Verification and generation of good function summaries is extensively discussed and widely used in software verification. We believe writing a good function summary requires strong domain knowledge in particular robot devices as well as the problem to be solved.

5 CASE STUDY: DISTRIBUTED FORMATION CONTROL

In this section, we revisit the LineForm program of Section 2. As mentioned in Section 4.2, the symbolic post configuration generated by $\mathbb{K}$ represents a set of *system configurations*. For variables in $\boldsymbol{c}$, their *primed copies*, and their double primed copies represent the variables in $\boldsymbol{c}' \in C^+$, and $\boldsymbol{c}'' \in (C^+_{\mathsf{prog}})_{[0,\delta]}$ respectively. Consider a candidate invariant:

Invariant 2. *:*

$$[\![I_i]\!]_{\boldsymbol{c}} := Motion.psn_i \in rect(\mathsf{p}_{min}, \mathsf{p}_{max}) \wedge x[i] \in rect(\mathsf{p}_{min}, \mathsf{p}_{max})$$

This invariant asserts that the position of each robot i is always within $rect(\mathrm{p}_{min}, \mathrm{p}_{max})$, and that each agent updates its shared variable value to one within $rect(\mathrm{p}_{min}, \mathrm{p}_{max})$ as well.

We first try to prove Invariant 2 without any assumptions, only from the constraints generated through the symbolic execution of LineForm. *Koord* Prover symbolically executes the event *TargetUpdate* (for robot i) and automatically generates the post event configuration constraint E_i :

$$E_i := \left(\begin{array}{l} \neg(i = N_{sys} - 1 \vee i = 0) \\ \wedge\ Motion.target'_i = (x[i-1] + x[i+1])/2 \wedge x'[i] = Motion.psn_i \\ \wedge\ u_vars \wedge Motion.psn''_i := traj(Motion.psn'_i, Motion.target_i, t) \wedge t \in [0, \delta] \end{array} \right)$$

where *traj* is treated as an uninterpreted function over $\mathbb{R} \times \mathbb{R}$. Note that the function *rect* can both be precisely defined as well as left uninterpreted. The primed copies of the variables in $\boldsymbol{c}$ are their values in $\boldsymbol{c}'$, and the double primed copies are their values in $\boldsymbol{c}''$. The rest of the formula includes a subformula *u_vars* that ensures the values of unmodified variables are unchanged such as $Motion.psn'_i = Motion.psn_i$ and $x'[j] = x[j]$ for $j \neq i$.

Since there is only one event, the induction proof obligation, *Koord* Prover generates the following proof obligation PO (1) for LineForm:

Proof Obligation 1. $\bigwedge_{i \in \mathtt{ID}} [\![I_i]\!]_{\boldsymbol{c}} \wedge E_i \Rightarrow [\![I_i]\!]_{\boldsymbol{c}''}$

The Prover returns that the negation of PO (1) is satisfiable, meaning that our proposed invariant is not inductive. The satisfying assignment serves as a counter example. This is not surprising as automatically generated proof obligations do not include any sensor or actuator assumptions. Specifically, PO (1) does not contain any restrictions on $Motion.psn''_i, Motion.target''_i$ w.r.t any of the variables in the symbolic post configuration after the event.

Next, we introduce a controller assumption $\langle P_i, Q_i \rangle$:

$$\begin{aligned} P_i &:= Motion.psn'_i \in rect(\mathrm{a}, \mathrm{b}) \wedge Motion.target'_i \in rect(\mathrm{a}, \mathrm{b}) \\ Q_i &:= Motion.psn''_i \in rect(\mathrm{a}, \mathrm{b}), \end{aligned} \tag{6}$$

where a and b are constants in $\mathbb{R}^3$, c' is the configuration P_i is evaluated on, and c'' is the configuration Q_i is evaluated on. PO (1) is then refined to:

Proof Obligation 2. $\bigwedge_{i \in \mathtt{ID}} [\![I_i]\!]_{\boldsymbol{c}} \wedge E_i \wedge (P_i \Rightarrow Q_i) \Rightarrow [\![I_i]\!]_{\boldsymbol{c}''}$

Having added the controller assumption (6), *Koord* Prover returns that the negation of PO (2) is unsatisfiable, i.e., (6) is sufficient to prove Invariant 2 . Table 1 summarizes the verification time needed for checking PO (1) on instances of LineForm with different N_{sys}.

N_{sys}	**dim**	T_K (s)	T_V (s)	**Safe**
3	1	4.90	9.09	✓
3	2	4.19	10.13	✓
4	1	4.79	12.21	✓
4	2	5.28	12.49	✓
4	3	5.06	12.77	✓
5	1	4.91	18.46	✓

N_{sys}	**dim**	T_K (s)	T_V (s)	**Safe**
5	2	5.60	18.91	✓
5	3	4.33	20.30	✓
10	1	4.92	32.34	✓
10	2	5.16	32.42	✓
10	3	4.34	33.61	✓
15	1	5.23	53.89	✓

Table 1. Summary of semantics based verification for LineForm. N is N_{sys}, T_K is the symbolic post computation time in $\mathbb{K}$, T_V is the time taken for construction of constraints and verification in Z3. Robots moving along a line are represented by **dim** = 1, along a plane by **dim** = 2, and in a 3D space by **dim** = 3.

We now turn to validating the controller assumption (Controller Assumption 6). Recall, from PO (*AAsm*) and $\langle P_i, Q_i \rangle$ above, we can derive the following:

Controller Proof Obligation 1.

$$\forall t \in [0, \delta], Motion.psn_i \in rect(\mathrm{a}, \mathrm{b}) \wedge Motion.target_i \in rect(\mathrm{a}, \mathrm{b}) \\ \wedge c'' = traj(c', t) \Rightarrow Motion.psn_i'' \in rect(\mathrm{a}, \mathrm{b})$$

It states that if the current position and the target of the robot are within the rectangle $rect(\mathrm{a}, \mathrm{b})$, then it remains within the rectangle for the next δ interval.

To prove (6), one obviously has to use the dynamics of the specific robot and the specific of waypoint-tracking controller that is implemented. Several approaches are available for checking this type of properties for control systems, such as, barrier certificates [Prajna and Jadbabaie 2004], reachability analysis [Asarin et al. 2000; Kurzhanski and Varaiya 2000], and Lyapunov analysis [Branicky 1998; Duggirala and Mitra 2012]. For each of these approaches there are many available algorithms and tools. In this paper, we use the reachability and in the remainder of this section, we give an overview of this analysis.

Reachability analysis computes the set of states of a control system that is reachable from a set of initial states. Proving Controller Proof Obligation 1 boils down to computing the set of reachable states from a set of initial positions bounded by $rect(\mathrm{a}, \mathrm{b})$ and with the target also in the same rectagle, and checking that the result is contained in $rect(\mathrm{a}, \mathrm{b})$. Typically, computing the exact set of reachable states is undecidable for nonlinear control system models, and therefore, the available algorithms rely on over-approximations

In this case study, we use the DryVR [Fan et al. 2017] reachability analysis tool which uses numerical simulations to learn the sensitivity of the trajectories of the robot. Then, it uses this sensitivity and additional simulations to either prove the required property, with a probabilistic guarantee, or finds a counter-example trace. DryVR has been used to analyze automotive and aerospace control systems [Fan et al. 2018]. Here we use the *Koord* simulator to generate traces of a quadcopter using the Hector quadrotor model [Meyer et al. 2012], from which DryVR computes the reachsets.

Figure 10 shows the outputs of the reachability analysis performed on quadcopters models. With a simple PID controller, the quadcopter overshoots its target, and violates the Controller Proof Obligation 1, while for the same controller with different control gains with a lower settling time, it meets the requirement. Here we have computed reach sets from a smaller initial rectangle and with a target that is also in a smaller rectangle, than $rect(\mathrm{a}, \mathrm{b})$. However, the model of the quadcopter, we note, is symmetric under translations, planar reflections and rotations. Therefore, using Theorem 10 from [Russo and Slotine 2011] and as shown in [Sibai et al. 2020], the computed reachsets can be translated and rotated to cover all initial and target choices in $rect(\mathrm{a}, \mathrm{b})$.

6 CASE STUDY : DISTRIBUTED DELIVERY

Delivery in Figure 11 is a *Koord* program to solve distributed delivery problem. The problem statement is as follows: Given a set of (possibly heterogeneous) robots, a safety distance $\epsilon > 0$, and a fixed sequence of delivery points (or tasks) $list = x_1, x_2, \ldots \in \mathbb{R}^3$, there are following two requirements: (a) every unvisited x_i in the sequence is *visited* exactly by one robot and (b) no two robots ever get closer than ϵ.

In this case study, we only prove requirement (b) for Delivery. Delivery consists of two events (1) *Assign*, in which each robot looks for an unassigned task x from *list*; if there is a clear path to x then the robot assigns itself the task x, set the actuator port *Motion.path*, and shares its path with all other robots thru *shared_path*. Otherwise, it shares its position as the path. (2) *Complete*, which checks whether an robot has visited its assigned task. A path here is a list of points that a robot visits in sequence. The *Motion* module drives the robot along a path, as directed by the position value set at its actuator port *Motion.path*. The sensor port *Motion.planner* returns a path to the

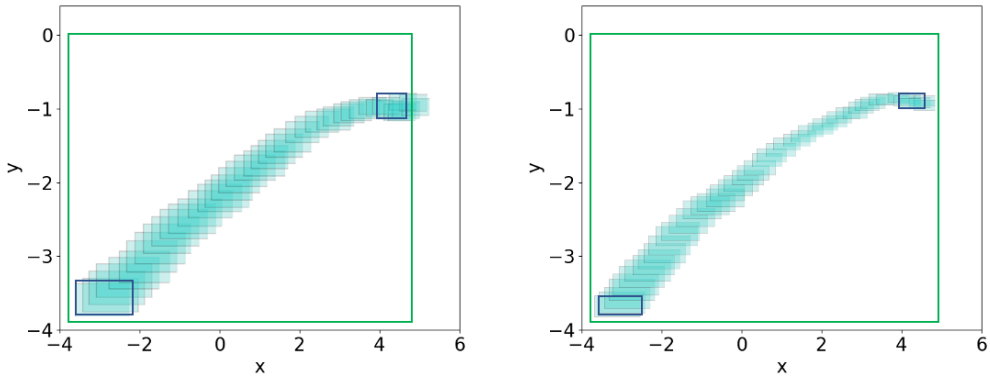

Fig. 10. Reachtube computations for lineform, for the quadcopter. The green rectangle represents *rect*(a, b). The blue rectangle at the bottom left corner of each plot represents starting points in the simulated trajectories used to generate these reachtubes, and the blue rectangle on the top right corner is the bound on the targets reached in the trajectories.*Left* shows that the reachtube of the quadcopter overshoots the rectangle. *Right* shows that with different PID control parameters, the controller assumption is satisfied.

```
using Motion:
 actuators:
  List⟨Point⟩ path
 sensors:
  Point position
  bool reached
  PathPlanner planner
local:
 bool on_task = s
 List⟨Point⟩ curr_path
 Task cur_task

allread:
 List⟨Point⟩ shared_paths[N_sys]
allwrite:
 List⟨Task⟩ all_tasks

Complete:
pre: on_task and Motion.reached
eff: on_task=False
shared_paths[pid]=[Motion.position]

Assign:
 pre: !on_task
 eff:
  if len(all_tasks) == 0:
   stop
  else: atomic:
   for t in all_tasks:
    curr_path=Motion.planner(t.target)
    if pathIsClear(shared_paths, \
              curr_path, pid):
     on_task=True
     cur_task=t
     break
   if on_task:
    all_tasks.remove(cur_task)
    shared_paths[pid]=curr_path
    Motion.path=curr_path
   else:
    shared_paths[pid]=[Motion.position]
```

Fig. 11. *Koord* code for distributed Delivery application.

target of an unassigned task, and a (user-defined) function called *pathIsClear* is used to determine whether the currently planned path is within ϵ distance of any path in *shared_path*.

Suppose there is a function parameterized by ϵ, taking two paths as input $clear_\epsilon : List\langle Point\rangle \times List\langle Point\rangle \mapsto bool$, it returns true only if the minimum distance between the two paths is greater than ϵ. We restate requirement (b) as:

Invariant 3. *No two robots ever get closer than ϵ. This gives us the following invariant for the current configuration c:*

$$[\![I_i]\!]_c = \forall j \in [N_{sys}], (i \neq j \wedge clear_\epsilon(shared_path[i], shared_path[j])) \vee (i = j)$$

Computing the *clear* function involves nested loops over the length of each path, then computing the minimum distance between each path segment *pathIsClear* further has to iterate over all shared paths and check via *clear*. We use the notion of *function summary* as defined in Section 4 to capture the notion of correctness for for *pathIsClear*. The function summary *PIC* is defined below as:

Function Summary 1.

$$PIC(sp, cp, i, y) := \forall j \in \mathtt{ID}, j \neq i \wedge \neg clear_\rho(sp[j], cp) \Rightarrow \neg y$$

where $0 \leq \rho < \epsilon$. The function summary simply says, if my current path cp is not more than ϵ distance to any path $sp[j]$ shared by other robots, the output of $y = pathIsClear(sp, cp, i)$ should be false,[3] and PO (*FSum*) becomes:

Proof Obligation 3.

$$\forall sp, cp, i, PIC(sp, cp, i, pathIsClear(sp, cp, i))$$

Validating PO (3) requires reasoning about the implementation of the *pathIsClear* function, which is beyond the scope of this discussion.

For constructing the symbolic set of configurations, we use a list with four tasks signified by $\{t_1, t_2, t_3, t_4\}$ so that the symbolic execution terminates. The for-loop iterating through the task list is unrolled into a sequence of (nested) *if-else* statements. For simplicity, we show the *automatically generated* symbolic post event configuration of the *Assign* event for only one execution when robot i picks t_1:

$$\begin{aligned} E_i^{t_1} := {} & \neg on_task_i \wedge on_task_i' \\ & \wedge curr_path_i' = M.planner(t_1.target) \\ & \wedge PIC(shared_path, curr_path_i', i, \mathtt{True}) \\ & \wedge shared_path'[i] = curr_path_i{}' \\ & \wedge M.path_i' = shared_path'[i] \wedge u_vars \end{aligned}$$

where u_vars again, ensures the values of unmodified variables are unchanged. Notice how we can use *PIC* to summarize *pathIsClear*. Similarly, we get $E_i^{t_2}$, $E_i^{t_3}$ and $E_i^{t_4}$ for other execution paths choosing corresponding tasks. When none of the tasks is picked, the post event configuration generated is

$$E_i^{none} := \neg on_task_i \wedge shared_path'[i] = [M.pos_i] \wedge u_vars$$

For the event *Assign*, the post event configuration is:

$$E_i := \left(\begin{array}{l} \forall j \in [N_{sys}], E_i^{t_1} \wedge E_i^{t_2} \wedge E_i^{t_3} \wedge E_i^{t_4} \wedge E_i^{none} \\ \wedge (M.pos'', M.reached'') = traj(M.pos', M.reached', M.path', t) \wedge t \in [0, \delta] \end{array} \right)$$

Our prover then automatically generates the proof obligation :

Proof Obligation 4. $\bigwedge_{i \in \mathtt{ID}} [\![I_i]\!]_c \wedge E_i \Rightarrow [\![I_i]\!]_{c''}$

[3]The index i in the *pathIsClear* function indicates that the previously shared path of the robot i shouldn't be considered in the computation.

Benchmark	N_{sys}	T_K (s)	T_V (s)	Safe
Task	3	9.90	10.6	✓
Task	4	9.79	11.78	✓
Task	5	9.91	14.92	✓
Task	10	12.92	18.34	✓

Table 2. Summary of semantics based verification for Delivery. T_K is the symbolic post computation time in $\mathbb{K}$, T_V is the time taken for generation of constraints and verification in Z3 and N_{sys} is the number of robots in the system.

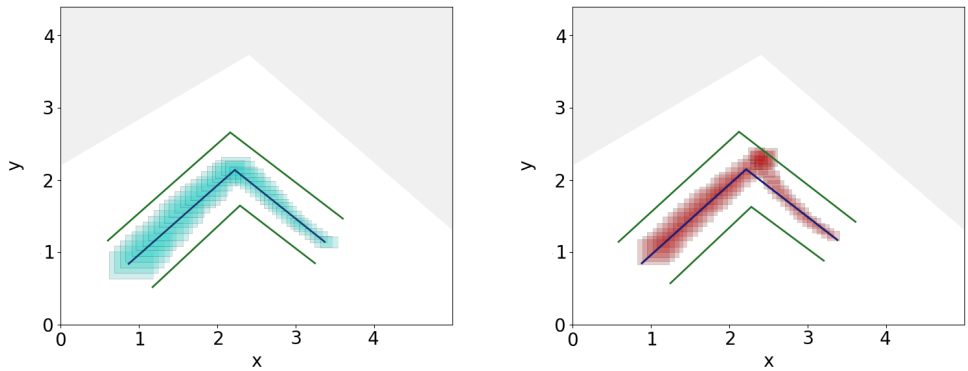

Fig. 12. Reachtube computations for Delivery. In both the plots, the grey shaded area is *unsafe* and needs to be avoided. The blue path is the computed path, and the green lines indicate the bounds at β distance from the path. *Left* shows the computed reachtube for the quadcopter lies within β of the actual path, thus the vehicle will not violate Controller Proof Obligation 2.*Right* shows the computed reachtube for our car model is not contained so the vehicle may violate the assumption.

Without any assumptions on robot motion, our tool is unable to discharge this proof automatically. For abstracting the movement of robots, an robot should move closely ($\neg clear_\beta$, where $\beta + \rho \leq \epsilon$) along its *Motion.path* actuator whose value is denoted by $M.path$ until it finishes traversing the path. For simplicity, we denote *Motion.reached* by $M.reached$. We add $\langle P_i, Q_i \rangle$ with

$$P_i := \neg M.reached_i'$$
$$Q_i := \neg clear_\beta(M.pos_i'', M.path_i'')$$

The corresponding proof obligation then becomes:

Controller Proof Obligation 2.

$$\forall t \in [0, \delta], \neg M.reached_i' \wedge c'' = traj(c', t) \Rightarrow \neg clear_\beta(M.pos_i'', M.path_i'')$$

The induction hypothesis for event *Complete* is generated similarly (omitted here), and the overall proof obligation is a conjunction of the two. Table 2 summarizes the verification of these constraints with different number of robots.

We now turn towards DryVR based validation for Controller Proof Obligation 2. To do this, we computed reachtubes for our vehicle models and checked whether they were contained within β distance of the desired path. We found that the reachtube of the quadcopter satisfied this requirement, but our car model did not, as seen in Figure 12 *(Right)*. The car-model [Karaman et al. 2017] we used has non-holonomic constraints, and making the turn formed by the two components of the

```
using MotionWithScan
  sensors:
    Point position
    List⟨Point, Scan⟩ pscan
    bool reached
    PathPlanner planner
  actuators:
    List⟨Point⟩ path

allwrite:
  GridMap map

#omitting initialization
local:
  GridMap localMap
  Point target
  bool on_path = True
  List⟨Grid⟩ obstacles

GUpdate:
  pre MotionWithScan.reached
  eff: atomic:
   map = merge(map, localMap)
   on_path = False

NewTarget:
  pre !on_path
  eff:
    target = pickFrontierPos(map, MotionWithScan.position)
    obstacles = findObs(map)
    MotionWithScan.path = MotionWithScan.planner(target, obstacles)
    if MotionWithScan.path != []:
      on_path = True
    else:
      on_path = False
    localMap = map

LUpdate:
  pre on_path and !MotionWithScan.reached
  eff:
    for p, s in MotionWithScan.pscan:
      localMap = merge(localMap, scanToMap(p, s))
```

Fig. 13. *Koord* code for Distributed Mapping Application Mapping

path shown in Figure 12 requires the car to perform a reverse maneuver that may violate the safety constraint.

7 CASE STUDY: DISTRIBUTED MAPPING

We demonstrate how *Koord* port abstractions can allow a versatility of robotic functionality through a distributed grid mapping application (Mapping). This problem requires a set of robots to collaboratively mark the position of static *obstacles* within a given area D quantized by a *grid*, which any robot should avoid while moving in D. This problem is a simplified version of the distributed Simultaneous Localization and Mapping problem (SLAM), a classical problem in robotics research. The difference comes with the assumption that the robots know their *global coordinates* within the area of deployment, and only attempt to map static obstacles within this area. Further, the only sensors available for sensing obstacles are LIDAR-based, and the robots are constrained to move in a 2D space. The *Lidar.ldata* sensor module is used to read the LIDAR scan of the actual robot.

The Mapping algorithm works in the following manner. Each robot constructs a *local grid map* over D using sensors, and updates it using information from other robots shared via a *global grid map*. In Figure 13, the *MotionWithScan* module provides a *pscan* sensor used to read the LIDAR scan of the actual robot. The other sensors and actuators *position, reached, planner, path* have the same functionality as that in the *Motion* module. The shared allwrite variable *map* is used to construct a shared map of obstacles within the domain D, and has type *GridMap*, which is a 2-D array representing a grid over D. The local variable *localMap* represents each robot's *local* knowledge of the domain D, and has the same type as D. There are three *events*: *NewPoint, LUpdate*, and *GUpdate*. A robot executing the *NewPoint* event, finds an unoccupied point to move to using a user defined function *pickFrontierPos* and plans a path to it using *MotionWithScan.planner*. It then updates its *localMap* from the shared variable *map*. The *LUpdate* event updates the *localMap* with scanned sensor data while the robot is in motion, and the *GUpdate* event updates the shared *map* with the updated *localMap* information corresponding to the scanned data.

For our case study, we tested the DMap application using the simulator from the *Koord* programming tools as implemented in [Author 2019], and chose the MIT RACECAR model [Karaman et al. 2017] included with the *Koord* vehicle models in the tools. Figure 14 shows the stages of the collaborative map created by 4 robots of the U-shaped obstacle in the simulation environment.

While tools such as ROS [Quigley et al. 2009] can be used to implement applications such as these, without inherent support for distributed coordination, it becomes difficult to program these applications even for experienced roboticists. Our *Koord* language helps provide easy integration with popular robotics platforms through platform specific implementations of controller abstractions, while providing support for the distributed coordination in the language itself.

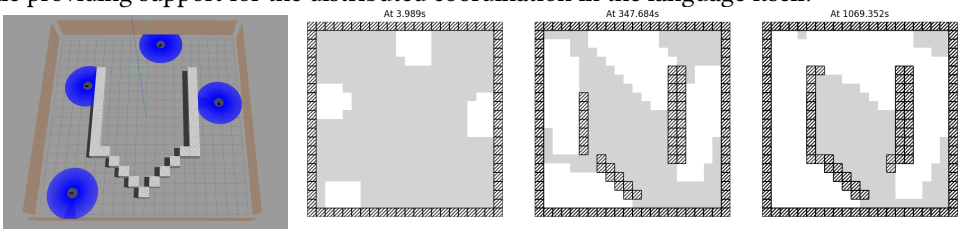

Fig. 14. Four cars with a U-shape world in the gazebo simulator environment provided by *Koord* programming tools (*Left*). Visualization of the global map at three different time stamps (*Right*)

8 RELATED WORK

Early domain specific languages for robotics were proprietary and tied to specific platforms. See [Nordmann et al. 2014] for a detailed survey. With the lowering hardware costs and increasing popularity, there is a growing interest in open and portable frameworks and languages [Bohrer et al. 2018; Pinciroli et al. 2015; Williams et al. 2003; Zufferey 2017]. *Robot Operating System (ROS)* [Quigley et al. 2009] is the predominant member in this category. At its core, ROS supports a publish-subscribe-based communication, and the ROS community has built drivers for numerous hardware components. Our implementation of the *Koord* abstractions for the quadcopter and vehicle platforms use ROS just like thousands of other robotics products and projects. The table above gives a summary of robotics languages that have been deployed on hardware.

Framework /system	**Dist. Sys.**	**Hetero-geneous**	**Sim**	**Prog. Lang.**	**Compiler**	**V&V**
ROSBuzz [St-Onge et al. 2017]	✓	✓	✓	Buzz	✓	
PythonRobotics		✓	✓	Python		
PyRobot [Murali et al. 2019]		✓	✓	Python		
MRPT [Blanco 2009]		✓		C++		
Robotarium [Pickem et al. 2017]		✓	✓	Matlab		
DRONA [Desai et al. 2017]	✓		✓	P [Desai et al. 2013]	✓	✓
Live [Campusano and Fabry 2017]		✓		LPR	✓	
Koord	✓	✓	✓	*Koord*	✓	✓

ROSBuzz [St-Onge et al. 2017] supports the Buzz language, which doesn't provide abstractions like *Koord* for path planning and shared variables. The Live Robot Programming language [Campusano and Fabry 2017] provides abstractions in terms of nested state machines and allows the program to be changed while running. It does not support robot ensembles. Programming systems using the shared memory paradigm have been developed for several distributed computing systems [Adve and Gharachorloo 1996; Calder et al. 2011; DeCandia et al. 2007; Lakshman and Malik 2010; Nitzberg and Lo 1991]. A position paper [Ghosh et al. 2018] proposed combining shared memory with physical interactions in a high-level language. Starting from a similar core idea, this paper presents a full language, develops its formalization, and the proof system that combines those abstractions.

P [Desai et al. 2013] and PSync [Drăgoi et al. 2016] are DSLs for asynchronous partially distributed systems, but cyber-physical interactions are not supported. P has been integrated into the DRONA framework [Desai et al. 2017] and the latter has very similar objectives to our work, but the approaches and solutions are different. DRONA is a framework for multi-robot motion planning and so far deployed only on drones. Koord and the underlying middleware aims to be more general, and multiple applications have been deployed on cars and drones in both simulations and hardware. The explicit model checker (using Zing) of DRONA relies on manual proofs of their safe-plan-generator and path-executor, which are analogous to Koord function summaries and controller assumptions. DRONA's model checker explores reachable states upto a given depth (number of transitions from an initial state). Koord proves inductive invariants using our own symbolic executable semantics. Therefore, when all proof obligations are discharged for a candidate invariant, the Koord system proves the invariant holds for all reachable states. Further, while our Task application implements something similar to the distributed plan generator which is a built-in feature for DRONA, *Koord*'s port interfaces allow portability across arbitrary planners.

[Author 2019] presents the CyPhyHouse toolchain, including overview of the whole software stack on which Koord runs, with the focus on deploying Delivery application (2nd case study) on vehicle and quadcopter hardware. In contrast, this paper presents the formal description of *Koord* language and the key language and verification challenges.

9 CONCLUSIONS AND FUTURE WORK

Our case studies with *Koord* demonstrate that DRAs with sensing, actuation, path planning, collision avoidance, and multi-robot coordination, can be succinct and amenable to formal analysis. The *Koord* programmer only needs to understand *Koord*'s shared memory semantics, and the sensor and actuator port abstractions. On the other hand, the hardware engineer will need to validate that the port abstractions are indeed met by the target hardware platform through testing. The symbolic execution of *Koord* programs can effectively automate analysis of inductive invariants of the distributed coordination logic. Distributed robotics applications may have nondeterministic behaviors. We found that inductive invariants, which were preserved during program transitions across *every event execution by any agent*, can be completely verified by our approach.

Further, the framework allows one to plug-in reachability analysis to validate/falsify controller assumptions for platform-dependent controllers. We performed case studies on applications that have been deployed on robots using *Koord*), and demonstrated how our semantics enables separate formal analyses using the prover for the distributed coordination and discrete programming logic, and DryVR for simulation driven verification of of the platform dependent control.

It is difficult to expect that any language, including controller assumptions, can cover vastly different types of robots (which are constantly evolving). To that end, our design on top of $\mathbb{K}$ semantics framework gives a flexible way to to extend our language and tailor it to specific robot types on demand. At the same time, it automatically adapts to the same framework for formal analysis. We plan to investigate this further on actual robots with diverse sensing and actuation capabilities. We also plan to extend our work to include specification and verification of progress properties under fairness constraints for *Koord* applications.

REFERENCES

Sarita V. Adve and Kourosh Gharachorloo. 1996. Shared memory consistency models: A tutorial. *IEEE Computer* 29 (1996), 66–76.

Rajeev Alur and David L. Dill. 1994. A theory of timed automata. *Theoretical Computer Science* 126 (1994), 183–235.

E. Asarin, O. Bournez, T. Dang, and O. Maler. 2000. Approximate Reachability Analysis of Piecewise-Linear Dynamical Systems. In *Hybrid Systems: Computation and Control (LNCS)*, B. Krogh and N. Lynch (Eds.), Vol. 1790. Hybrid Systems: Computation and Control, 20–31.

Hagit Attiya and Jennifer Welch. 2004. *Distributed Computing: Fundamentals, Simulations and Advanced Topics.* John Wiley & Sons, Inc., USA.

Anonymous Author. 2019. Anonymous Paper. In *to appear in ICRA 2020*. IEEE, Paris,France, 6 pages.

Stanley Bak and Parasara Sridhar Duggirala. 2017. Hylaa: A tool for computing simulation-equivalent reachability for linear systems. In *Proceedings of the 20th International Conference on Hybrid Systems: Computation and Control*. ACM, 173–178.

José-Luis Blanco. 2009. *Contributions to Localization, Mapping and Navigation in Mobile Robotics.* Ph.D. Dissertation. PhD. in Electrical Engineering, University of Malaga. http://www.mrpt.org/Paper:J.L._Blanco_Phd_Thesis

Timo Blender, Thiemo Buchner, Benjamin Fernandez, Benno Pichlmaier, and Christian Schlegel. 2016. Managing a Mobile Agricultural Robot Swarm for a seeding task. In *IECON 2016-42nd Annual Conference of the IEEE Industrial Electronics Society*. IEEE, 6879–6886.

Brandon Bohrer, Yong Kiam Tan, Stefan Mitsch, Magnus O. Myreen, and André Platzer. 2018. VeriPhy: Verified Controller Executables from Verified Cyber-physical System Models. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*. ACM, New York, NY, USA, 617–630. DOI:http://dx.doi.org/10.1145/3192366.3192406

M. Branicky. 1998. Multiple Lyapunov functions and other analysis tools for switched and hybrid systems. *IEEE Trans. Automat. Control* 43 (1998), 475–482.

Brad Calder, Ju Wang, Aaron Ogus, Niranjan Nilakantan, Arild Skjolsvold, Sam McKelvie, Yikang Xu, Shashwat Srivastav, Jiesheng Wu, Huseyin Simitci, Jaidev Haridas, Chakravarthy Uddaraju, Hemal Khatri, Andrew Edwards, Vaman Bedekar, Shane Mainali, Rafay Abbasi, Arpit Agarwal, Mian Fahim ul Haq, Muhammad Ikram ul Haq, Deepali Bhardwaj, Sowmya Dayanand, Anitha Adusumilli, Marvin McNett, Sriram Sankaran, Kavitha Manivannan, and Leonidas Rigas. 2011. Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles (SOSP '11)*. ACM, New York, NY, USA, 143–157. DOI:http://dx.doi.org/10.1145/2043556.2043571

Miguel Campusano and Johan Fabry. 2017. Live Robot Programming: The Language, its Implementation, and Robot API Independence. *Science of Computer Programming* 133 (2017), 1 – 19.

Xin Chen, Erika Ábrahám, and Sriram Sankaranarayanan. 2013. Flow*: An analyzer for non-linear hybrid systems. In *Computer Aided Verification*. Springer, 258–263.

Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. 2007. Dynamo: Amazon's Highly Available Key-value Store. In *Proceedings of Twenty-first ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*. ACM, New York, NY, USA, 205–220. DOI:http://dx.doi.org/10.1145/1294261.1294281

Ankush Desai, Vivek Gupta, Ethan Jackson, Shaz Qadeer, Sriram Rajamani, and Damien Zufferey. 2013. P: Safe Asynchronous Event-driven Programming. *SIGPLAN Not.* 48, 6 (June 2013), 321–332. DOI:http://dx.doi.org/10.1145/2499370.2462184

Ankush Desai, Indranil Saha, Jianqiao Yang, Shaz Qadeer, and Sanjit A Seshia. 2017. Drona: A framework for safe distributed mobile robotics. In *2017 ACM/IEEE 8th International Conference on Cyber-Physical Systems (ICCPS)*. IEEE, 239–248.

Cezara Drăgoi, Thomas A. Henzinger, and Damien Zufferey. 2016. PSync: A Partially Synchronous Language for Fault-tolerant Distributed Algorithms. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '16)*. ACM, New York, NY, USA, 400–415. DOI:http://dx.doi.org/10.1145/2837614.2837650

Parasara Sridhar Duggirala and Sayan Mitra. 2012. Lyapunov Abstractions for Inevitability of Hybrid Systems. In *The 15th International Conference on Hybrid Systems: Computation and Control (HSCC 2012), Beijing, China*.

Parasara Sridhar Duggirala, Sayan Mitra, and Mahesh Viswanathan. 2013. Verification of Annotated Models from Executions. In *EMSOFT*.

Chuchu Fan, Bolun Qi, and Sayan Mitra. 2018. Data-Driven Formal Reasoning and Their Applications in Safety Analysis of Vehicle Autonomy Features. *IEEE Design & Test* 35, 3 (2018), 31–38. DOI:http://dx.doi.org/10.1109/MDAT.2018.2799804

Chuchu Fan, Bolun Qi, Sayan Mitra, and Mahesh Viswanathan. 2017. DryVR: Data-driven verification and compositional reasoning for automotive systems. In *Computer Aided Verification (CAV)*.

Goran Frehse, Colas Le Guernic, Alexandre Donzé, Scott Cotton, Rajarshi Ray, Olivier Lebeltel, Rodolfo Ripado, Antoine Girard, Thao Dang, and Oded Maler. 2011. SpaceEx: Scalable Verification of Hybrid Systems. In *CAV (Lecture Notes in Computer Science)*, Ganesh Gopalakrishnan and Shaz Qadeer (Eds.), Vol. 6806. Springer, 379–395.

David Gauthier, Paul Freedman, Gregory Carayannis, and Alfred Malowany. 1987. Interprocess communication for distributed robotics. *IEEE Journal on Robotics and Automation* 3, 6 (1987), 493–504.

Mario Gerla, Eun-Kyu Lee, Giovanni Pau, and Uichin Lee. 2014. Internet of vehicles: From intelligent grid to autonomous cars and vehicular clouds. In *2014 IEEE world forum on internet of things (WF-IoT)*. IEEE, 241–246.

Ritwika Ghosh, Sasa Misailovic, and Sayan Mitra. 2018. Language Semantics Driven Design and Formal Analysis for Distributed Cyber-Physical Systems: [Extended Abstract]. In *Proceedings of the 2018 Workshop on Advanced Tools, Programming Languages, and PLatforms for Implementing and Evaluating Algorithms for Distributed Systems (ApPLIED '18)*. ACM, New York, NY, USA, 41–44. DOI:http://dx.doi.org/10.1145/3231104.3231958 event-place: Egham, United

Kingdom.

Lars Grüne and Jürgen Pannek. 2017. Nonlinear model predictive control. In *Nonlinear Model Predictive Control*. Springer, 45–69.

Ge Guo and Wei Yue. 2012. Autonomous platoon control allowing range-limited sensors. *IEEE Transactions on vehicular technology* 61, 7 (2012), 2901–2912.

Thomas A. Henzinger, Peter W. Kopke, Anuj Puri, and Pravin Varaiya. 1995. What's decidable about hybrid automata?. In *ACM Symposium on Theory of Computing*. 373–382. citeseer.nj.nec.com/henzinger95whats.html

S. Karaman, A. Anders, M. Boulet, J. Connor, K. Gregson, W. Guerra, O. Guldner, M. Mohamoud, B. Plancher, R. Shin, and J. Vivilecchia. 2017. Project-based, collaborative, algorithmic robotics for high school students: Programming self-driving race cars at MIT. In *2017 IEEE Integrated STEM Education Conference (ISEC)*. 195–203. DOI:http://dx.doi.org/10.1109/ISECon.2017.7910242

Alexander B. Kurzhanski and Pravin Varaiya. 2000. Ellipsoidal Techniques for Reachability Analysis. In *HSCC*. 202–214. citeseer.csail.mit.edu/kurzhanski98ellipsoidal.html

Michal Kvasnica, Pascal Grieder, Mato Baotić, and Manfred Morari. 2004. Multi-parametric toolbox (MPT). In *Hybrid systems: computation and control*. Springer, 448–462.

Avinash Lakshman and Prashant Malik. 2010. Cassandra: a decentralized structured storage system. *ACM SIGOPS Operating Systems Review* 44, 2 (2010), 35–40.

Steven M LaValle. 1998. Rapidly-exploring random trees: A new tool for path planning. (1998).

Nancy A. Lynch. 1996. *Distributed Algorithms*. Morgan Kaufmann Publishers Inc., Cambridge.

Johannes Meyer, Alexander Sendobry, Stefan Kohlbrecher, Uwe Klingauf, and Oskar von Stryk. 2012. Comprehensive Simulation of Quadrotor UAVs Using ROS and Gazebo. In *Simulation, Modeling, and Programming for Autonomous Robots*, Itsuki Noda, Noriaki Ando, Davide Brugali, and James J. Kuffner (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 400–411.

Pieter J Mosterman, David Escobar Sanabria, Enes Bilgin, Kun Zhang, and Justyna Zander. 2014. A heterogeneous fleet of vehicles for automated humanitarian missions. *Computing in Science & Engineering* 16, 3 (2014), 90.

Adithyavairavan Murali, Tao Chen, Kalyan Vasudev Alwala, Dhiraj Gandhi, Lerrel Pinto, Saurabh Gupta, and Abhinav Gupta. 2019. PyRobot: An Open-source Robotics Framework for Research and Benchmarking. *arXiv preprint arXiv:1906.08236* (2019).

B. Nitzberg and V. Lo. 1991. Distributed shared memory: a survey of issues and algorithms. *Computer* 24, 8 (aug. 1991), 52–60. DOI:http://dx.doi.org/10.1109/2.84877

Arne Nordmann, Nico Hochgeschwender, and Sebastian Wrede. 2014. *A Survey on Domain-Specific Languages in Robotics*. Springer International Publishing, Cham, 195–206. DOI:http://dx.doi.org/10.1007/978-3-319-11900-7_17

D. Pickem, P. Glotfelter, L. Wang, M. Mote, A. Ames, E. Feron, and M. Egerstedt. 2017. The Robotarium: A remotely accessible swarm robotics research testbed. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*. 1699–1706. DOI:http://dx.doi.org/10.1109/ICRA.2017.7989200

Carlo Pinciroli, Adam Lee-Brown, and Giovanni Beltrame. 2015. Buzz: An Extensible Programming Language for Self-Organizing Heterogeneous Robot Swarms. *CoRR* abs/1507.05946 (2015). http://arxiv.org/abs/1507.05946

J Norberto Pires and JMG Sá Da Costa. 2000. Object-oriented and distributed approach for programming robotic manufacturing cells. *Robotics and computer-integrated manufacturing* 16, 1 (2000), 29–42.

Andr Platzer. 2018. *Logical Foundations of Cyber-Physical Systems* (1st ed.). Springer Publishing Company, Incorporated.

Stephen Prajna and Ali Jadbabaie. 2004. Safety verification of hybrid systems using barrier certificates. In *In Hybrid Systems: Computation and Control*. Springer, 477–492.

Morgan Quigley, Ken Conley, Brian P. Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng. 2009. ROS: an open-source Robot Operating System. In *ICRA Workshop on Open Source Software*. IEEE.

Redmond R Shamshiri, Cornelia Weltzien, Ibrahim A Hameed, Ian J Yule, Tony E Grift, Siva K Balasundram, Lenka Pitonakova, Desa Ahmad, and Girish Chowdhary. 2018. Research and development in agricultural robotics: A perspective of digital farming. (2018).

Grigore Rosu and Traian Florin Serbanuta. 2014. K Overview and SIMPLE Case Study. In *Proceedings of International K Workshop (K'11) (ENTCS)*, Vol. 304. Elsevier, Illinois, 3–56. DOI:http://dx.doi.org/10.1016/j.entcs.2014.05.002

Giovanni Russo and Jean-Jacques E Slotine. 2011. Symmetries, stability, and control in nonlinear systems and networks. *Physical Review E* 84, 4 (2011), 041929.

Hussein Sibai, Navid Mokhlesi, Chuchu Fan, and Sayan Mitra. 2020. Multi-Agent Safety Verification using Symmetry Transformations. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 173–190.

David St-Onge, Vivek Shankar Varadharajan, Guannan Li, Ivan Svogor, and Giovanni Beltrame. 2017. ROS and Buzz: consensus-based behaviors for heterogeneous teams. *CoRR* abs/1710.08843 (2017). http://arxiv.org/abs/1710.08843

Sebastian Thrun and others. 2002. Robotic mapping: A survey. *Exploring artificial intelligence in the new millennium* 1, 1-35

(2002), 1.
Brian C Williams, Michel D Ingham, Seung H Chung, and Paul H Elliott. 2003. Model-based programming of intelligent embedded systems and robotic space explorers. *Proc. IEEE* 91, 1 (2003), 212–237.
Damien Zufferey. 2017. The REACT language for robotics. (2017).